



PyTorch is an open-source deep learning library, originally developed by Meta Platforms and currently developed with support from the Linux Foundation.

Pytorch

Outline

- Introduction
- How PyTorch Works?
 - Data types
 - Functions
 - Differentiation in Autograd
 - PyTorch with GPU's
 - Neural Networks
- Learning PyTorch with Examples

Introduction

- PyTorch is an open source machine learning framework that accelerates the path from research to production
- Developed primarily by Facebook AI and introduced in 2016



Introduction

- The framework combines the efficient GPU-accelerated backend libraries from Torch with Python frontend
 - Focuses on rapid prototyping, readable code, and support for the variety of deep learning models
- It allows deep learning models to be expressed in the Python programming language

Introduction

- Most data scientists are familiar with Python
 - PyTorch stayed within this ecosystem to keep the interfaces simple and consistent
- It hid much of the complexity inherent to machine learning behind intuitive APIs
 - Deep Learning primitives
 - Layer types
 - Activation & Losses and optimizers etc.

Introduction

- Pytorch allowed researchers to manually control the execution of their code to provide pragmatic performance
- PyTorch implementation was designed to be simple but slightly “incomplete”
 - As opposed to more comprehensive and complex frameworks
 - This makes it easier to implement additional features and adapt to new situations in PyTorch
 - Increasing its flexibility

Introduction

- PyTorch applies a graph-metaprogramming based approach
 - The executive code for defining layers, composing models, loading data and running optimizer is all expressed by general purpose programming
 - Any new neural network architecture can be easily generalized and implemented with PyTorch

Performance of PyTorch

- The PyTorch core is used to implement tensor data structure, CPU and GPU operators, basic parallel primitives and automatic differentiation calculations
 - As the most intensive computing operations are handled by the core, they can be written in the efficient C++ programming language to boost performance
- A strict separation is maintained between control and data flow
 - Although the control flow is in Python, the optimized C++ code can be executed on the host CPU

Important Properties of PyTorch



- Python support
 - PyTorch is based on Python, it can be used with popular libraries and packages such as NumPy, SciPy, Numba and Cython
 - Offers developers an easy-to-learn, simple-to-code structure that's based on Python
 - Enables easy debugging with popular Python tools
- TorchScript
 - Production environment of PyTorch that enables users to transition between modes
 - TorchScript optimizes functionality, speed, ease of use and flexibility

Important Properties of PyTorch

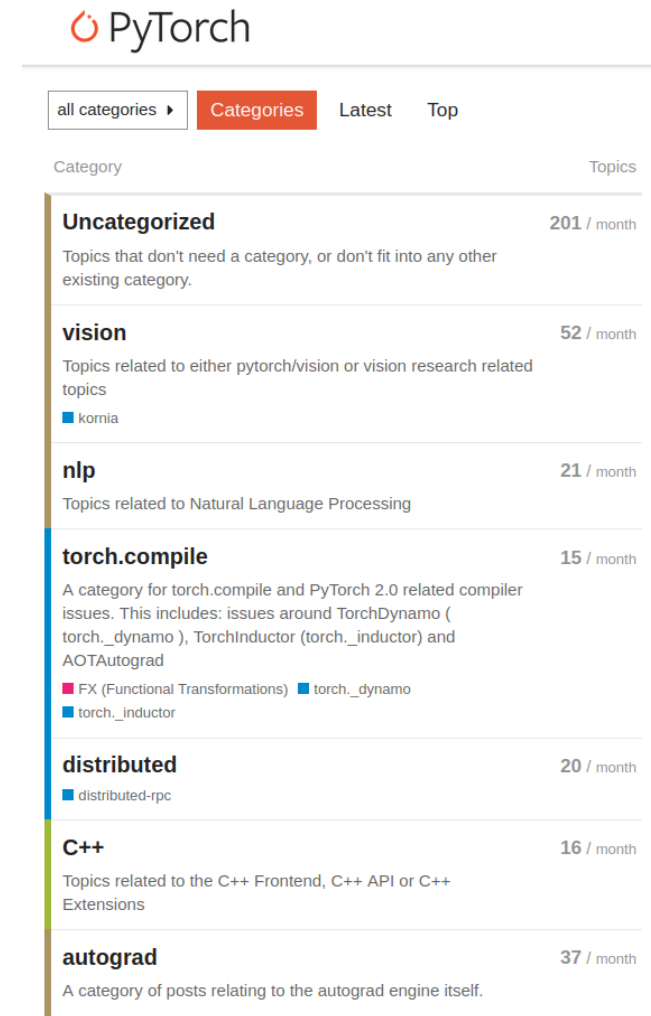
- Dynamic graph computation
 - Lets users change network behavior on the fly, rather than waiting for all the code to be executed
- Automatic differentiation
 - Used for creating and training neural networks.
 - It numerically computes the derivative of a function by making backward passes in neural networks

Important Properties of PyTorch

- Offers scalability and is well-supported on major cloud platforms
- It supports CPU, GPU, and parallel processing, as well as distributed training
- The [PyTorch Hub](#) is a repository of pre-trained models that can be invoked, in some cases with just a single line of code
 - It has a large collection of tools and libraries in areas ranging from computer vision to reinforcement learning

Important Properties of PyTorch

- Provides a small community focused on open source
 - [PyTorch.org](https://pytorch.org) community with excellent documentation and tutorials. The forums are active and supportive



The screenshot shows the PyTorch forum categories page. At the top is the PyTorch logo. Below it are navigation tabs: 'all categories' (selected), 'Categories', 'Latest', and 'Top'. The main content is a table of forum categories with a vertical bar on the left indicating the number of topics in each category.

Category	Topics
Uncategorized Topics that don't need a category, or don't fit into any other existing category.	201 / month
vision Topics related to either pytorch/vision or vision research related topics ■ kornia	52 / month
nlp Topics related to Natural Language Processing	21 / month
torch.compile A category for torch.compile and PyTorch 2.0 related compiler issues. This includes: issues around TorchDynamo (torch._dynamo), TorchInductor (torch._inductor) and AOTAutograd ■ FX (Functional Transformations) ■ torch._dynamo ■ torch._inductor	15 / month
distributed ■ distributed-rpc	20 / month
C++ Topics related to the C++ Frontend, C++ API or C++ Extensions	16 / month
autograd A category of posts relating to the autograd engine itself.	37 / month

How to Download

- For installation, first, you have to choose your preference and then run the install command
- From pytorch.org/get-started/locally you can install by following instructions

PyTorch Build	Stable (2.1.2)		Preview (Nightly)	
Your OS	Linux	Mac	Windows	
Package	Conda	Pip	LibTorch	Source
Language	Python		C++ / Java	
Compute Platform	CUDA 11.8	CUDA 12.1	ROCm 5.6	CPU
Run this Command:	<pre>pip3 install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu118</pre>			

How PyTorch Works? - Tensors

- Fundamentally, it's a library for programming with tensors
- Tensors are the fundamental building blocks of neural networks in PyTorch
- Tensors are a specialized data structure that are very similar to arrays and matrices
 - Which are basically just multidimensional arrays!
 - In PyTorch, we use tensors to encode the inputs and outputs of a model, as well as the model's parameters
- Tensors are similar to [NumPy's](#) ndarrays, except that tensors can run on GPUs

Tensors

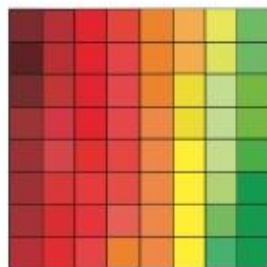
tensor = multidimensional array

vector



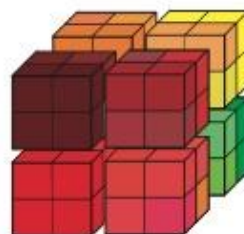
$$\mathbf{v} \in \mathbb{R}^{64}$$

matrix



$$\mathbf{X} \in \mathbb{R}^{8 \times 8}$$

tensor



$$\mathbf{X} \in \mathbb{R}^{4 \times 4 \times 4}$$

Tensors

- Tensors can be initialized in various ways
- The simplest way to create a tensor is with the **torch.empty()** call:

```
x = torch.empty(3, 4)
```

- Created tensor x is 2-dimensional, with 3 rows and 4 columns
- By default, PyTorch tensors are 32-bit floating numbers
- **torch.empty()** allocates memory for the tensor, but does not initialize it with any values

Tensors

- More often we'll want to initialize our tensor with some value
 - Common cases are:
 - All zeros
 - All ones
 - Random values
- torch module provides methods for all these!

```
zeros = torch.zeros(2, 3)
```

A tensor full of zeros

```
ones = torch.ones(2, 3)
```

A tensor full of ones

```
torch.manual_seed(1729)  
random = torch.rand(2, 3)
```

A tensor with random
values between 0 and 1

Tensors

- While initializing tensors, such as a model's learning weights, random values are common
 - But we need reproducibility of our results
 - Manually setting your random number generator (**torch.manual_seed()**) is the way to do this

Tensors

- Most of the time, when we are performing operations on more than one tensor, we need to have them of the same **shape**
 - Having the same number of dimensions, same number of cells in each dimension
- To initialize a tensor with the same shape as another tensor, we are using **torch.*_like()** methods:
 - `torch.empty_like(another_tensor)`
 - `torch.zeros_like(another_tensor)`
 - `torch.ones_like(another_tensor)`
 - `torch.rand_like(another_tensor)`

Tensors

- We can also specify the data of the tensor directly from a Pytorch collection:
- **torch.tensor()** is the most straightforward way to create a tensor if we already have data
- **torch.tensor()** creates a copy of the data
- Most of the time, our data starts out in NumPy arrays or pandas DataFrames
- We have to convert these data types to tensors using **torch.tensor()**
- **torch.tensor()** method takes two arguments: numerical data (NumPy array, Python list, or Python numeric variable) and desired data type (the dtype parameter)

Tensors

- We can also get the same tensor in our specified data type using methods such as `float()`, `long()` etc.
 - We can also use `tensor.FloatTensor`, `tensor.LongTensor`, `tensor.Tensor` classes to instantiate a tensor of particular type
- Or using **`.to()`**
- Available data types include:
 - `torch.bool`
 - `torch.int8`
 - `torch.uint16`
 - `torch.float`
 - `torch.double`

Data type	dtype
16-bit floating point [1]	<code>torch.float16</code> or <code>torch.half</code>
32-bit floating point	<code>torch.float32</code> or <code>torch.float</code>
64-bit floating point	<code>torch.float64</code> or <code>torch.double</code>
64-bit complex	<code>torch.complex64</code> or <code>torch.cfloat</code>
128-bit complex	<code>torch.complex128</code> or <code>torch.cdouble</code>
8-bit integer (unsigned)	<code>torch.uint8</code>
8-bit integer (signed)	<code>torch.int8</code>
16-bit integer (signed)	<code>torch.int16</code> or <code>torch.short</code>
32-bit integer (signed)	<code>torch.int32</code> or <code>torch.int</code>
64-bit integer (signed)	<code>torch.int64</code> or <code>torch.long</code>
Boolean	<code>torch.bool</code>

Tensors

- `torch.arange(end)`: Returns a 1-D tensor with elements ranging from 0 to end-1
 - We can use the optional start and step parameters to create tensors with different ranges

```
x = torch.arange(25).view((5, 5))
```



```
tensor([[ 0,  1,  2,  3,  4],  
        [ 5,  6,  7,  8,  9],  
        [10, 11, 12, 13, 14],  
        [15, 16, 17, 18, 19],  
        [20, 21, 22, 23, 24]])
```

Attributes of a Tensor

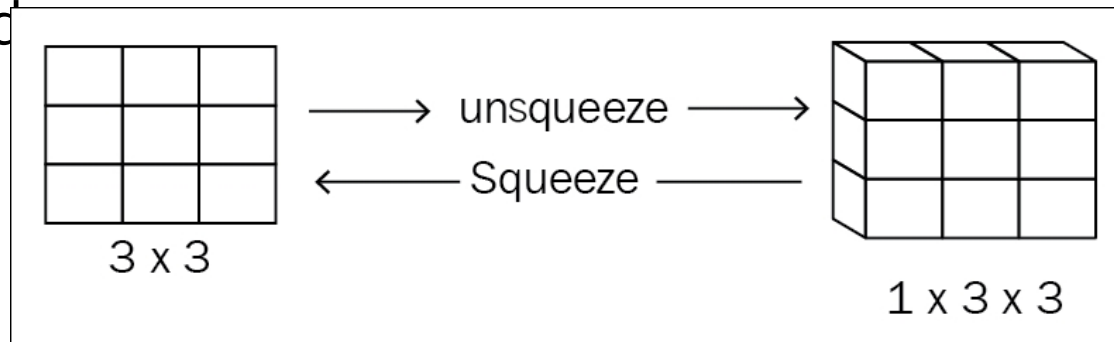
- Shape
- Datatype
 - Float32, Float64, Integer, Boolean
- Device
 - GPU/CPU

Properties of Tensor

- We can get the size of a particular dimension with the **size()** method
 - `x.size(0)` get's the size of 0th dimension
- Change the shape of a tensor with the **view()** method
 - `x_view = x.view(3, 2)` (`x_view` shares the same memory as `x`, so changing one changes the other)
- We can also use **torch.reshape()** method for a similar purpose
 - `x_reshaped = torch.reshape(x, (2, 3))`

Properties of Tensor

- We can use **torch.unsqueeze(x, dim)** function to add a dimension of size 1 to the provided dim
- We can also use the corresponding use **torch.squeeze(x)**, which removes the dimensions of size 1
- If we want to get the total number of elements in a tensor, we can use the **numel()** method



Operations

- There are over 100 operations
 - Arithmetic, linear algebra, matrix manipulation, sampling etc.
- Numpy-like indexing and slicing
 - In PyTorch we can index tensors, similar to NumPy
 - We can also index into multiple dimensions with :
 - `x[:, 0, 0]` gets the top left element of each element in our tensor
 - We can also access arbitrary elements in each dimension

Operations - Basic Operations

- PyTorch operations are very similar to those of NumPy
- We can work with both scalars and other tensors
 - $x + 2$ performs element-wise addition
 - $x * 2$ performs element-wise multiplication
- We can apply the same operations between different tensors of compatible sizes

Operations - Basic Operations

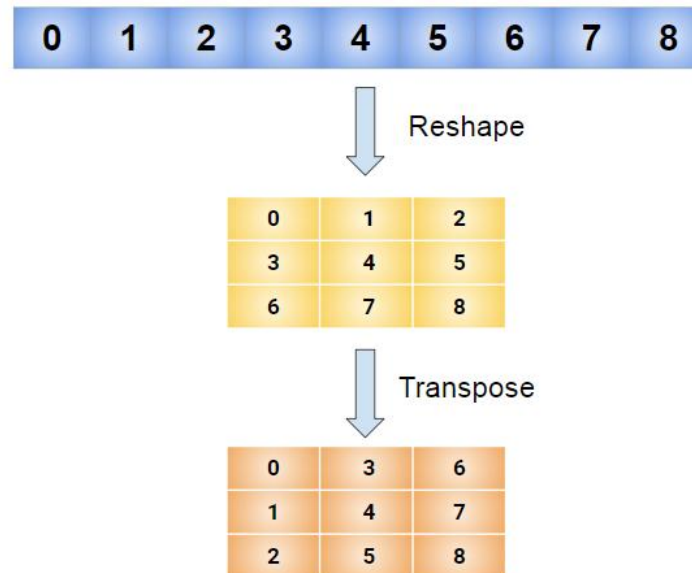
- It's important that if we want to apply a binary operation on tensors, they have to be identical shape
 - What happens when we try to perform a binary operation on tensors with dissimilar shapes?
 - Our code will throw a **run-time error**! (the size of tensor a must match the size of tensor b at non-singleton dimension)

Operations - Tensor Broadcasting

- The exception to the same-shape rule is tensor broadcasting
- Broadcasting is a way to perform an operation between tensors that have similarities in their shapes
- The rules for broadcasting are:
 - Each tensor must have at least one dimension - no empty tensors
 - Comparing the dimension sizes of the two tensors, going from last to first:
 - Each dimension must be equal, or
 - One of the dimensions must be of size 1, or
 - The dimension does not exist in one of the tensors

Operations – Matrix Multiplication

- We can use;
 - **tensor.matmul(other_tensor)** for matrix multiplication
 - Matrix multiplication can also be performed with **@**
 - **tensor.T** for transpose



Operations

- We can take the mean and standard deviation along a certain dimension with the methods **mean(dim)** and **std(dim)**
 - We can call these methods with no parameter to get the mean and standard deviation for the whole tensor
 - To use mean and std our tensor should be a floating-point type

```
# Create an example tensor
```

```
m = torch.tensor(  
    [  
        [1., 1.],  
        [2., 2.],  
        [3., 3.],  
        [4., 4.]  
    ]  
)
```

```
pp.pprint("Mean: {}".format(m.mean()))  
pp.pprint("Mean in the 0th dimension: {}".format(m.mean(0)))  
pp.pprint("Mean in the 1st dimension: {}".format(m.mean(1)))
```

'Mean: 2.5'

'Mean in the 0th dimension: tensor([2.5000, 2.5000])'

'Mean in the 1st dimension: tensor([1., 2., 3., 4.])'

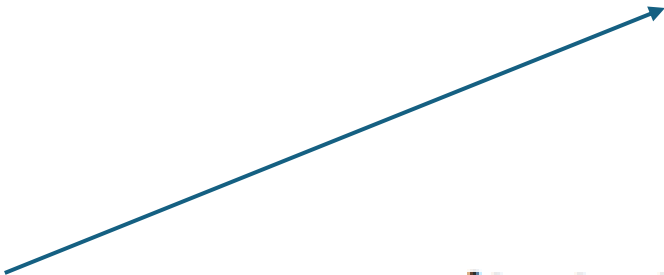
Operations

- We can concatenate tensors using **torch.cat**

```
# Concatenate in dimension 0 and 1
a_cat0 = torch.cat([a, a, a], dim=0)
a_cat1 = torch.cat([a, a, a], dim=1)

print("Initial shape: {}".format(a.shape))
print("Shape after concatenation in dimension 0: {}".format(a_cat0.shape))
print("Shape after concatenation in dimension 1: {}".format(a_cat1.shape))
```

```
Initial shape: torch.Size([4, 3])
Shape after concatenation in dimension 0: torch.Size([12, 3])
Shape after concatenation in dimension 1: torch.Size([4, 9])
```



[6., 6., 6.]
[6., 6., 6.]
[6., 6., 6.]
[6., 6., 6.]
[6., 6., 6.]
[6., 6., 6.]
[6., 6., 6.]
[6., 6., 6.]
[6., 6., 6.]
[6., 6., 6.]
[6., 6., 6.]
[6., 6., 6.]



[6., 6., 6.] [6., 6., 6.] [6., 6., 6.]
[6., 6., 6.] [6., 6., 6.] [6., 6., 6.]
[6., 6., 6.] [6., 6., 6.] [6., 6., 6.]
[6., 6., 6.] [6., 6., 6.] [6., 6., 6.]

Operations

- Most of the operations in PyTorch are not in place
- PyTorch offers the in-place versions of operations available by adding an **underscore** (**_**) at the end of the method name

```
# add() is not in place  
a.add(a)
```



```
tensor([[6., 6., 6.],  
        [6., 6., 6.],  
        [6., 6., 6.],  
        [6., 6., 6.]])
```

```
# add_() is in place  
a.add_(a)
```



```
tensor([[12., 12., 12.],  
        [12., 12., 12.],  
        [12., 12., 12.],  
        [12., 12., 12.]])
```

Copying Tensors

- Like Python, assigning a tensor to a variable makes the variable a label of the tensor, and does not copy it!
- What if we want a separate copy of the data?
 - You can use the `clone()` method

Moving to GPU

- One of the main advantages of PyTorch is its robust acceleration on CUDA-compatible Nvidia GPU's
- How do we move our work to a faster hardware?
 - First, we need to check whether a GPU is available using **`torch.cuda.is_available()`** method
 - If we have available GPU, we need to put our data to somewhere where the GPU can see it
 - CPU does the computation on data in RAM
 - GPU has dedicated memory attached to it
 - So, you must move all the data needed for computation to memory accessible by that device

Moving to GPU

- There are multiple ways:
 - You can do it at creation by defining device parameter
 - By default, tensors are created on CPU
 - If you have existing tensor, you can move it to another device (such as GPU or CPU) by using **.to('cuda')** or **.to('cpu')** method
 - If you are doing a computation involving two or more tensors, they should be on the same device

Break

Autograd

- PyTorch and other machine learning libraries are known for their automatic differentiation feature
- PyTorch adopted a Chainer innovation called reverse-mode automatic differentiation
- Essentially, it's like a tape recorder that records completed operations and then replays backward to compute gradients
- We can call the **backward()** method to ask PyTorch to calculate the gradients, which are then stored in the grad attribute

Autograd - Example

```
# Create an example tensor  
# requires_grad parameter tells PyTorch to store gradients  
x = torch.tensor([2.], requires_grad=True)
```

```
# Calculating the gradient of y with respect to x  
y = x * x * 3 # 3x^2  
y.backward()  
pp.pprint(x.grad) # d(y)/d(x) = d(3x^2)/d(x) = 6x = 12
```

```
tensor([12.])
```

Let's run backprop from a different tensor again to see what happens:

```
z = x * x * 3 # 3x^2  
z.backward()  
pp.pprint(x.grad)
```

```
tensor([24.])
```

We can see that the x.grad is updated to be the sum of the gradients calculated so far

Autograd

- A machine learning model is a function, with inputs and outputs
- Our output of interest will be model's loss, in context of training
- Since we want to minimize the loss, we are taking it's first derivative w.r.t. the input
- But we are not directly calculating loss from the input, so this derivative contains a lot of chain rules!

Autograd

- In our models, we have the partial derivatives of the model's outputs w.r.t. its inputs
- The full expression for each partial derivative is the sum of the products of the local gradients
 - In simpler words, the gradients over the learning weights

Autograd

- When we run backprop in a neural network, we sum up all the gradients for a particular neuron before making an update
- We need to run **zero_grad()** in every training iteration (more on this later)
- Otherwise, our gradients would keep building up from one training iteration to the other, which would cause our updates to be wrong

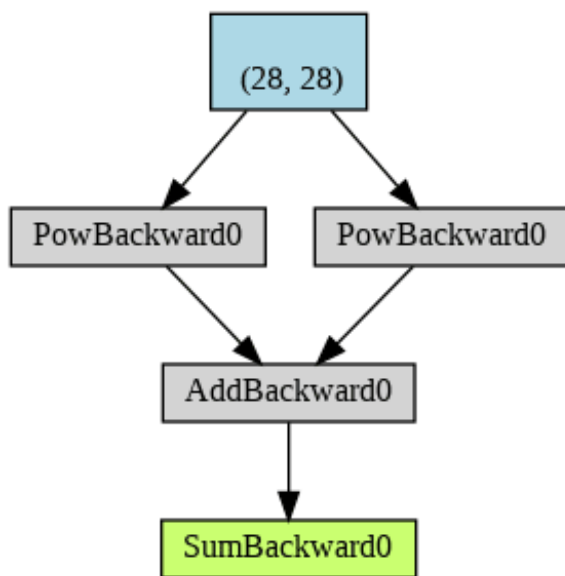
detach()

- Remember **torch.tensor()**
 - Constructs a tensor with no autograd history
- **tensor.clone()** creates a copy of tensor that imitates the original tensor's `requires_grad` field
- You should use **detach()** when attempting to remove a tensor from a computation graph

detach()

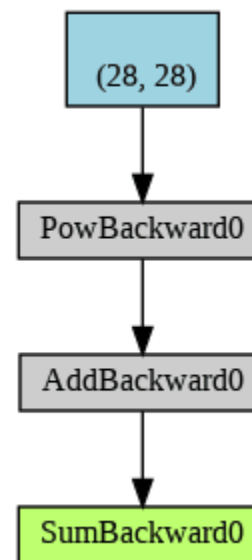
```
X = torch.ones((28, 28), dtype=torch.float32, requires_grad=True)
y = X**2
z = X**2

result = (y+z).sum()
```



```
X = torch.ones((28, 28), dtype=torch.float32, requires_grad=True)
y = X**2
z = X.detach()**2

result = (y+z).sum()
```

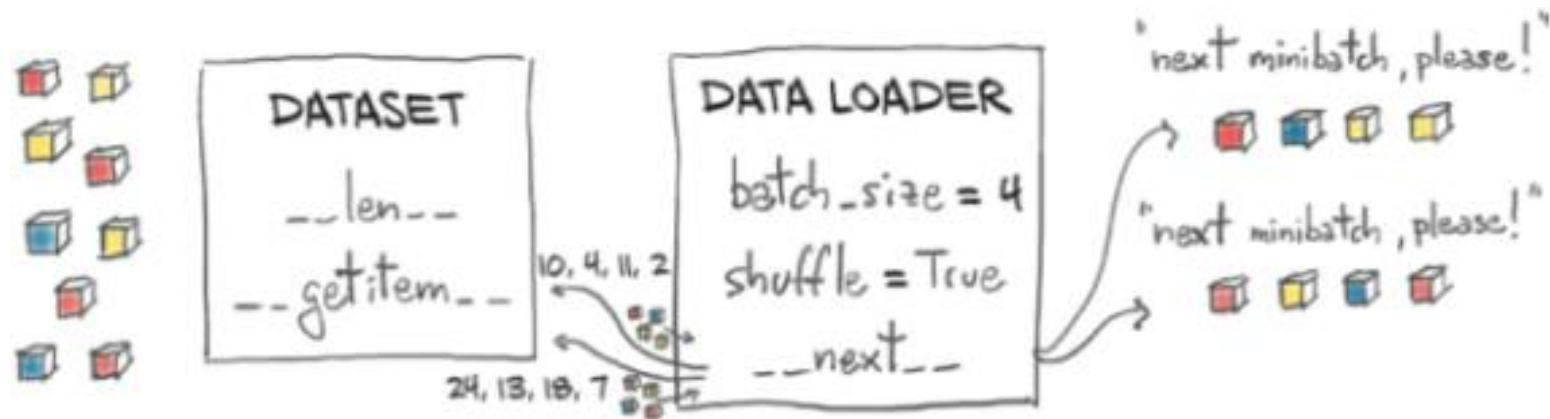


Datasets & DataLoaders

- Code for processing data samples can get messy and hard to maintain
- We ideally want our dataset code to be decoupled from our model training code for better readability and modularity
- PyTorch provides two data primitives:
 - **`torch.utils.data.DataLoader` & `torch.utils.data.Dataset`**
 - Allows us to use pre-loaded or own data

Datasets & DataLoaders

- Dataset stores the samples and their corresponding labels
- DataLoader wraps an iterable around the Dataset to enable easy access to the samples
- We can index Datasets manually like a list: `training_data[index]`

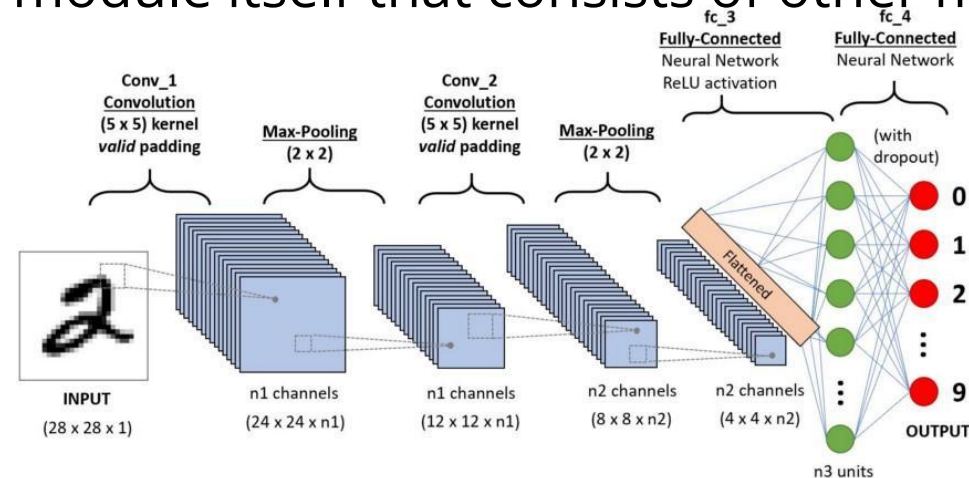


Datasets & DataLoaders

- PyTorch domain libraries provide a number of pre-loaded datasets (such as FashionMNIST) that subclass **torch.utils.data.Dataset** and implement functions specific to the particular data
- We can also create custom Dataset classes for own data
- A custom Dataset class must implement three functions:
 - The **__init__** function is run once when instantiating the Dataset object
 - The **__len__** function returns the number of samples in our dataset
 - The **__getitem__** function loads and returns a sample from the dataset at the given index idx

Building the Neural Network

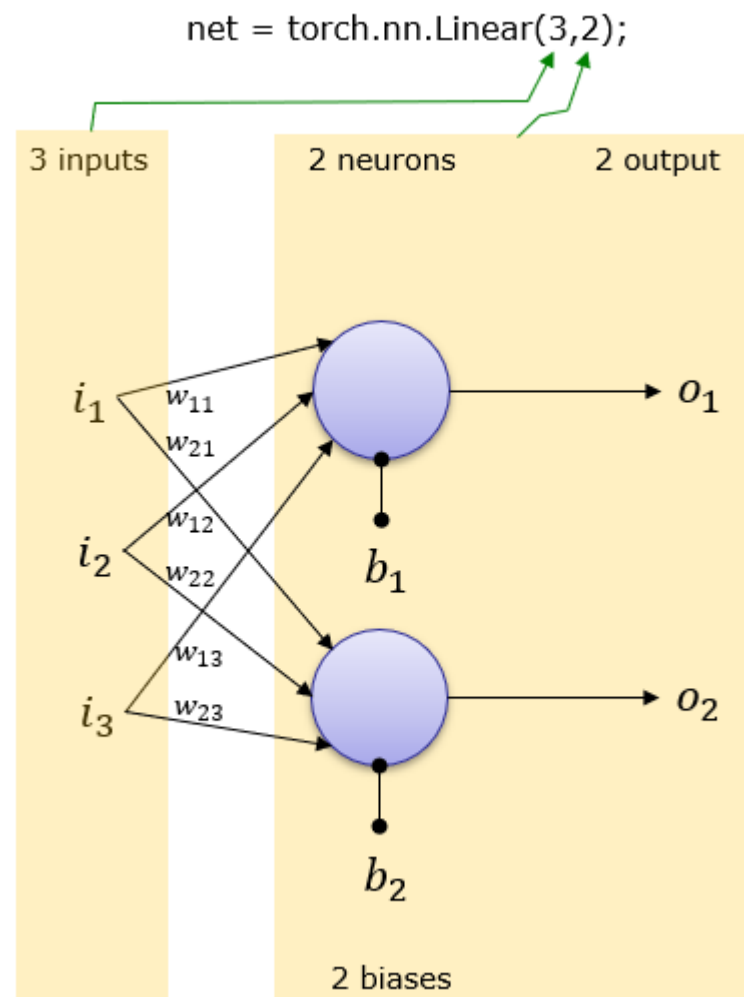
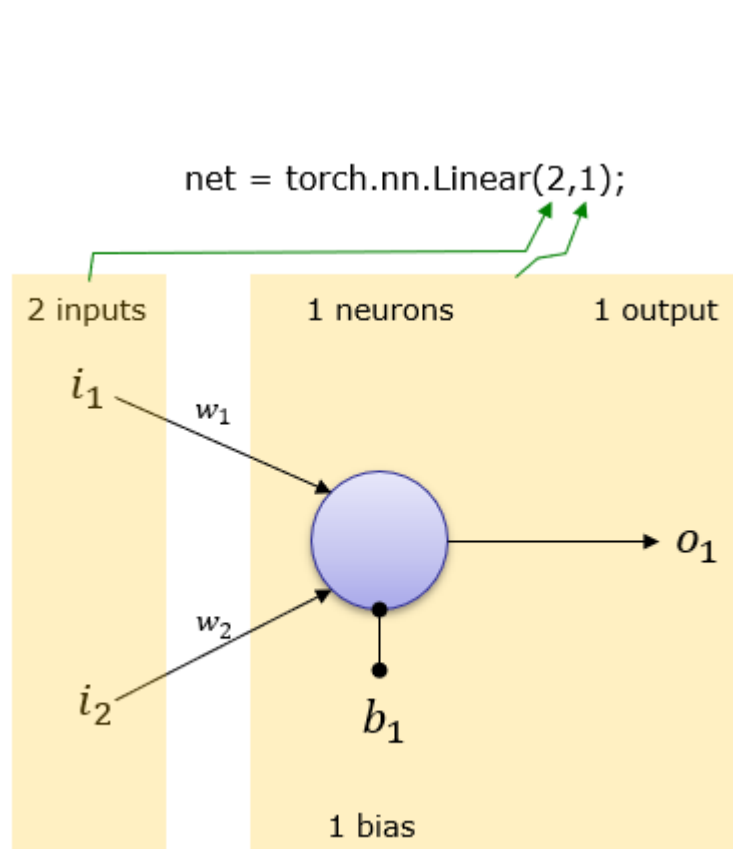
- Neural networks comprise of layers/modules that perform operations on data.
- [torch.nn](#) namespace provides all the building blocks you need to build your own neural network
 - Every module in PyTorch subclasses the [nn.Module](#)
- A neural network is a module itself that consists of other modules (layers)



Linear Layer

- The linear layer performs the operation $Ax+b$, where A and b are initialized randomly
- If we don't want the linear layer to learn the bias parameters, we can initialize our layer with `bias=False`
 - We can use **`nn.Linear(H_in, H_out)`** to create a linear layer
 - $(N, *, H_{in})$ dimensions and output a matrix of $(N, *, H_{out})$
 - The $*$ denotes that there could be arbitrary number of dimensions in between

Linear Layer



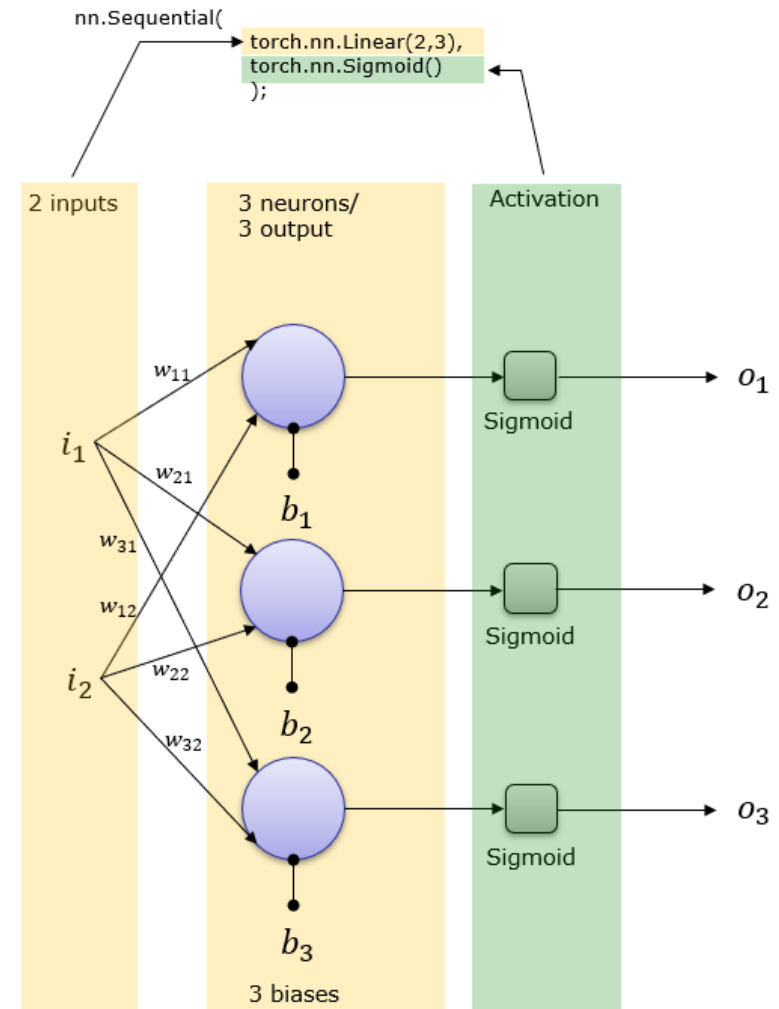
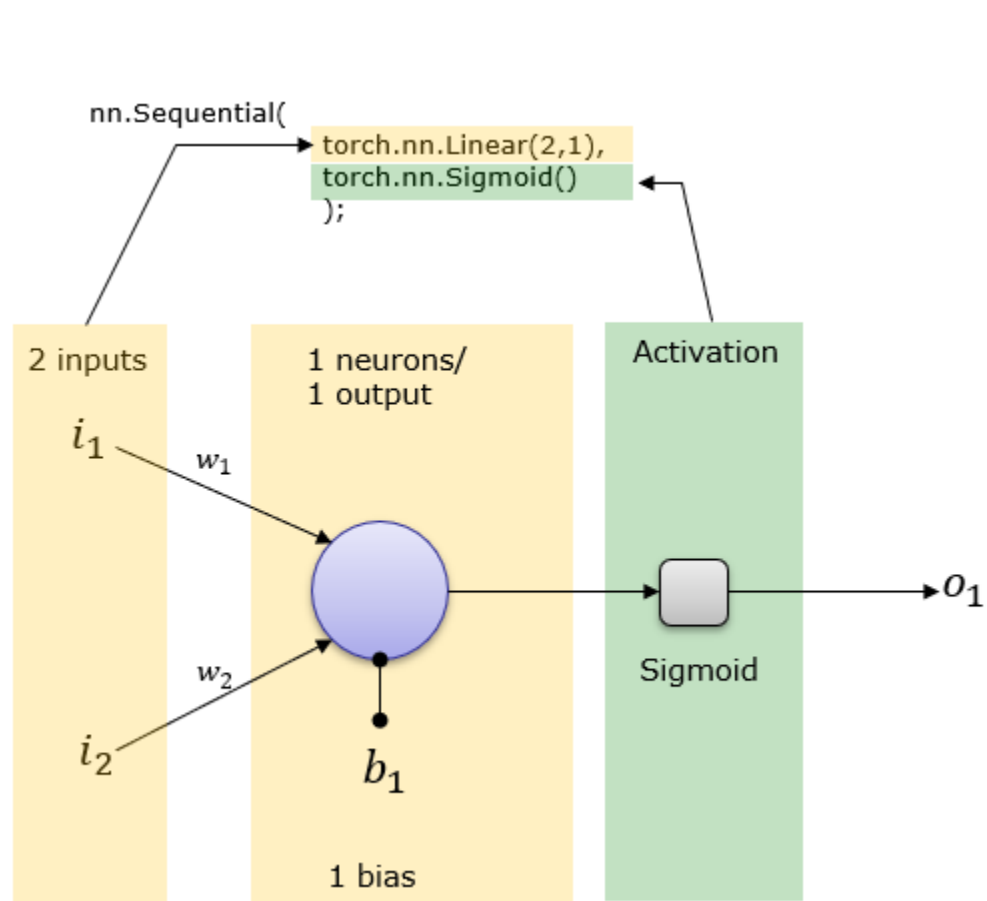
Other Layers

- There are several other preconfigured layers in the nn module
 - `nn.Conv2d`, `nn.ConvTranspose2d`, `nn.BatchNorm1d`, `nn.BatchNorm2d`,
`nn.Upsample`, `nn.MaxPool2d`
- We can treat each of these layers as plug and play components

Activation Layer

- Activation functions are used to add non-linearity to our network
- We can also use the nn module to apply activations functions to our tensors
 - **nn.ReLU(), nn.Sigmoid(), nn.LeakyReLU()**
- Activation functions operate on each element separately
 - Shape of the tensors we get as an output are the same as the ones we pass in

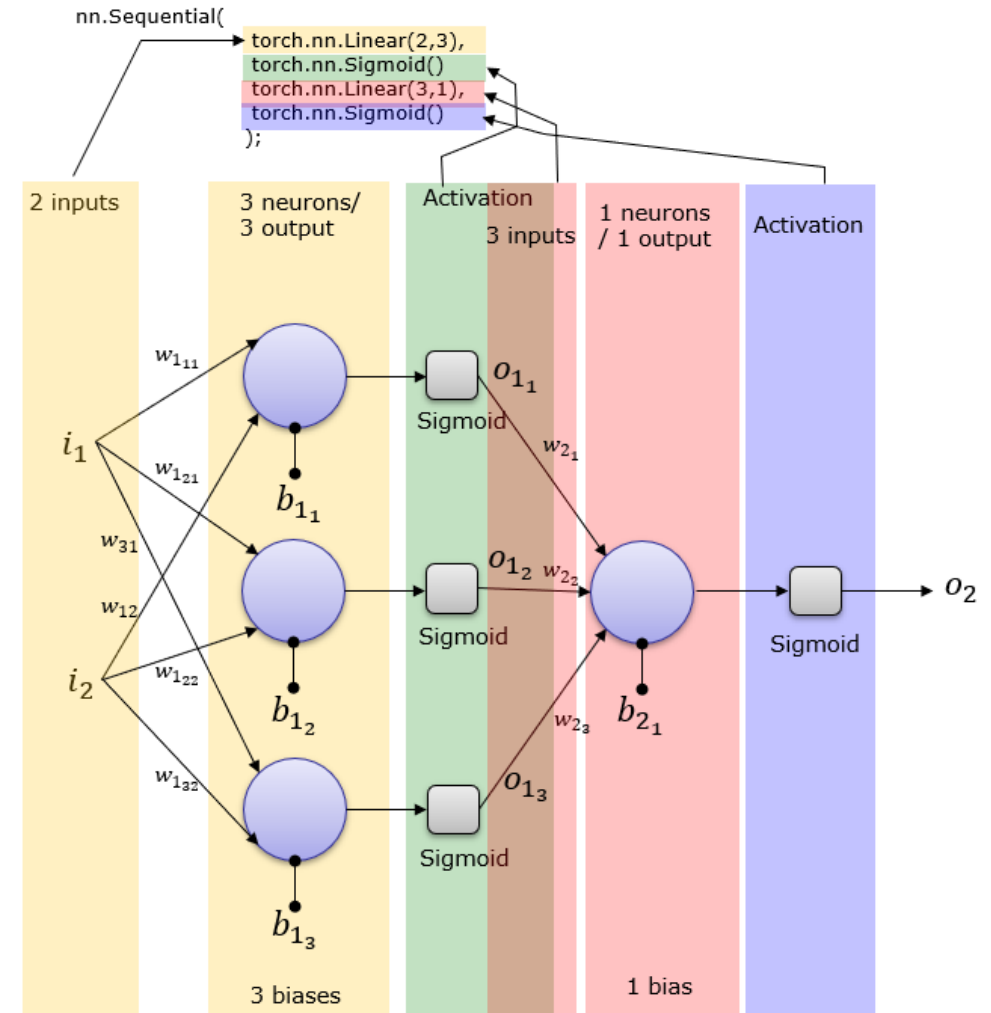
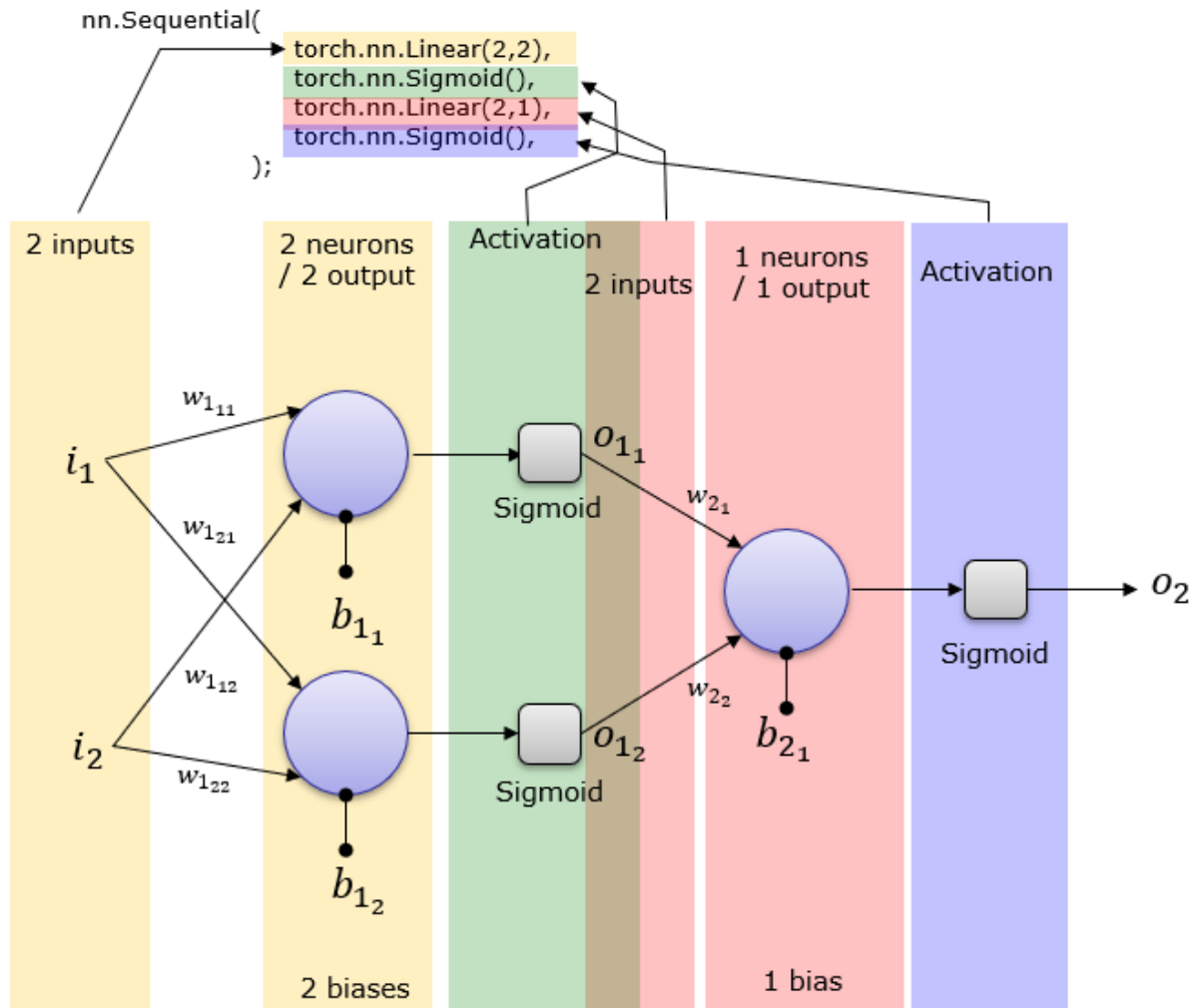
Activation Layer



Putting the Layers Together

- So far we have seen that we can create layers and pass the output of one as the input of the next
- Instead of creating intermediate tensors and passing them around, we can use **nn.Sequential**

Putting the Layers Together



Custom Modules

- Instead of using the predefined modules, we can also build our own by extending the `nn.Module` class
 - We can build a the `nn.Linear` (which also extends `nn.Module`) on our own using the tensor introduced earlier!
- We can also build new, more complex modules, such as a custom neural network

Custom Modules

- To create a custom module, the first thing we have to do is to extend the `nn.Module`
- We can then initialize our parameters in the `__init__` function, starting with a call to the `__init__` function of the super class
- All the class attributes we define which are `nn` module objects are treated as parameters, which can be learned during the training
- Tensors are not parameters, but they can be turned into parameters if they are wrapped in `nn.Parameter` class

Custom Modules

- All classes extending `nn.Module` are also expected to implement a `forward(x)` function, where `x` is a tensor
 - This is the function that is called when a parameter is passed to our module, such as in `model(x)`
- More on this in second part!

Optimization

- Having the gradients isn't enough for our models to learn
- We have showed how gradients are calculated with the **backward()** function
- We also need to know how to update the parameters of our models
- torch.optim module contains several optimizers that we can use
 - **optim.SGD** and **optim.Adam**
- Optimizers also has a learning rate (lr) parameter, which determines how big of an update will be made in every step
 - Different optimizers have different hyperparameters as well

Optimization

- After we have our optimization function, we can define a loss that we want to optimize for
- We can either define the loss ourselves or use one of the predefined loss function in PyTorch
 - such as `nn.BCELoss()`

PyTorch training loop

```
1 # Pass the data through the model for a number of epochs (e.g. 100)
2 for epoch in range(epochs):
3
4     # Put model in training mode (this is the default state of a model)
5     model.train()
6
7     # 1. Forward pass on train data using the forward() method inside
8     y_pred = model(X_train)
9
10    # 2. Calculate the loss (how different are the model's predictions to the true values)
11    loss = loss_fn(y_pred, y_true)
12
13    # 3. Zero the gradients of the optimizer (they accumulate by default)
14    optimizer.zero_grad()
15
16    # 4. Perform backpropagation on the loss
17    loss.backward()
18
19    # 5. Progress/step the optimizer (gradient descent)
20    optimizer.step()
```

Note: all of this can be turned into a function

Pass the data through the model for a number of **epochs** (e.g. 100 for 100 passes of the data)

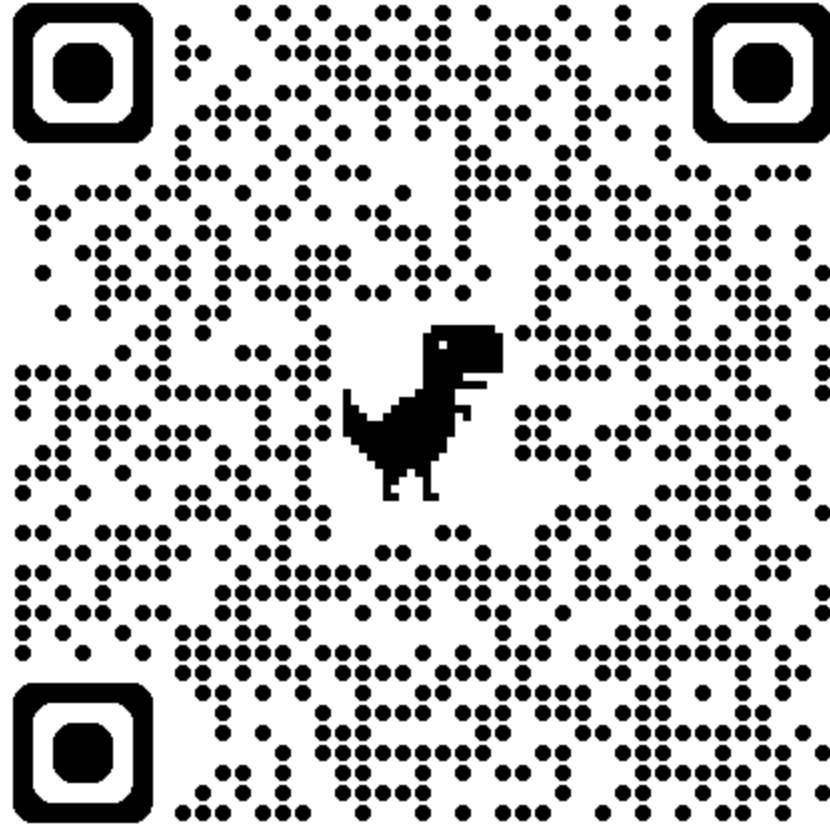
Pass the data through the model, this will perform the **forward()** method located within the model object

Calculate the loss value (how wrong the model's predictions are)

Zero the optimizer gradients (they accumulate every epoch, zero them to start fresh each forward pass)

Perform **backpropagation** on the loss function (compute the gradient of every parameter with `requires_grad=True`)

Step the optimizer to update the model's parameters with respect to the gradients calculated by `loss.backward()`



Scan For Colab Notebook